Aritmética Takum na Extensão Vetorial RISC-V: codificação, simulação funcional e avaliação de SpMV

Grupo do Projeto Integrador AOC – UFMG
Departamento de Engenharia Eletrônica
Belo Horizonte, MG, Brasil

***Resumo*—Extensões vetoriais modernas acumulam formatos de ponto flutuante de baixa precisão, cada um com conversões, casos especiais e decodificadores próprios. A aritmética Takum oferece precisão afunilada, faixa dinâmica limitada e uma decodificação comum de 12 bits. Este trabalho propõe a extensão experimental Zvtakum, sobre a RVV 1.0, com cinco instruções para T8, T16 e T32 codificadas no espaço custom-0. A aritmética é implementada em inteiros, com arredondamento para o par no código. Um simulador funcional RV64IM, Zicsr, RVV e Zvtakum, em C++17, foi verificado contra o QEMU, byte a byte, nas instruções padrão, e contra um modelo Python de racionais exatos, nas instruções Takum e nos kernels. Avaliamos o produto matriz-vetor esparso (SpMV) em oito matrizes Harwell–Boeing, com FP32 e Takum, para VLEN de 128 a 512 bits. As 96 execuções dos kernels são bit a bit iguais ao modelo de referência, e o número de instruções segue um modelo analítico que reproduz todas as execuções sem resíduo. Com acumuladores de 32 bits nos dois casos, o erro mediano do Takum32 é 2,01 vezes menor que o do FP32. Em nenhum dos 24 pares matriz×VLEN o kernel Takum32 executa mais instruções que o FP32, e as escritas de CSR são iguais. Em 8 e 16 bits, o Takum não produz resultados não finitos em nenhuma das matrizes, enquanto E4M3, E5M2 e FP16 estouram em três ou quatro delas. Onde nenhum dos formatos estoura, o resultado varia entre matrizes e formatos.**

***Palavras-chave*—RISC-V, RVV, aritmética Takum, Zvtakum, SpMV, simulador funcional, custom-0.**

I. Introdução

Os conjuntos de instruções vetoriais acumulam formatos de ponto flutuante de baixa precisão, como bfloat16, FP16 e os formatos OFP8 E4M3 e E5M2, voltados a inteligência artificial e computação de alto desempenho. Cada formato adiciona conversões, casos especiais e lógica de decodificação. A RVV 1.0 não define FP8, e o suporte a FP16 e bfloat16 depende de extensões separadas [1]–[3].

A aritmética Takum [4] é um formato de precisão afunilada. Seus códigos de n bits compartilham uma decodificação de 12 bits, a faixa dinâmica é limitada e não há infinito. Hunhold [5] mostrou, para o AVX10.2, que substituir formatos de ponto flutuante por Takum simplifica o conjunto de instruções. Este trabalho faz um estudo análogo para a RVV.

As contribuições são:

- uma codificação para cinco instruções Takum (`vtadd`, `vtsub`, `vtmul`, `vtdot` e `vncvt`) no espaço custom-0, verificada contra o inventário do riscv-opcodes;
- uma aritmética Takum em inteiros, sem `double`, com arredondamento para o par no código;
- um simulador funcional RV64IM, Zicsr, RVV e Zvtakum, verificado contra o QEMU e contra um modelo de racionais exatos;
- kernels SpMV em RISC-V para FP32 e Takum, com o mesmo perímetro de contagem e saída verificada bit a bit;
- um modelo analítico de contagem de instruções dos kernels, medido com programas sintéticos e conferido nas execuções reais;
- controles de acumulação que separam o efeito da largura do acumulador do efeito do formato.

O escopo é funcional. Não há síntese, área, frequência, energia nem tempo de execução. A variante logarítmica e a FP16 vetorial não são executadas nesta versão.

II. Fundamentação teórica

A. Takum linear

Um código Takum de n bits tem sinal S, direção D, três bits de regime R, r bits de característica C e $p = n - 5 - r$ bits de fração F, com $r = R$ se $D = 1$ e $r = 7 - R$ se $D = 0$. O valor é $(1 - 3S + F/2^p) \cdot 2^e$, com $e = (-1)^S(c + S)$. Para $n < 12$, a janela de decodificação tem 12 bits, com zeros à direita. O código $1000\ldots0$ é NaR, e os resultados fora da faixa são saturados.

B. Extensão vetorial RISC-V

A RVV 1.0 controla o comprimento dos vetores com `vtype` (SEW, LMUL, políticas de cauda e máscara) e `vl`. A instrução `vsetvli` escolhe `vl` como o mínimo entre o AVL e VLMAX. Com rs1 = x0 e rd ≠ x0, o AVL é VLMAX; com rs1 = rd = x0, o AVL é o `vl` atual. O gather indexado `vluxei32` lê elementos da memória a partir de índices de 32 bits.

C. SpMV em CSR

No formato CSR, a linha i da matriz ocupa as posições de `rowptr[i]` a `rowptr[i+1]`, com índices de coluna em `colidx`. O produto $y = Ax$ acumula, linha a linha, os produtos dos valores pelos elementos de x indicados por `colidx`.

Tabela I
CODIFICAÇÕES ZVTAKUM (CUSTOM-0). CAMPOS: FUNCT6 EM 31–26, VM EM 25, VS2 EM 24–20, VS1 EM 19–15, FUNCT3 EM 14–12 E VD EM 11–7.

funct6	Instrução	Operação
0	`vtadd.vv`	$vd_i = vs2_i + vs1_i$
1	`vtsub.vv`	$vd_i = vs2_i - vs1_i$
2	`vtmul.vv`	$vd_i = vs2_i \times vs1_i$
3	`vtdot.vv`	$vd_0 \leftarrow vd_0 + \sum_i vs2_i \times vs1_i$
4	`vncvt.t.t`	estreitamento de 2·SEW para SEW

III. TRABALHOS RELACIONADOS

Hunhold et al. [4], [5] definem o formato Takum e avaliam sua substituição no AVX10.2; a descrição de um decodificador em VHDL está em [6]. Posits [7], bfloat16 [3] e OFP8 [2] são os formatos de referência de baixa precisão. A biblioteca libtakum [8] é a implementação de referência em C. A ferramenta MuFoLAB [9] automatiza o benchmark de representação com 1401 matrizes; este trabalho não a executou (ver a seção de limitações). A especificação da RVV [1] define o comportamento padrão usado nas verificações, com QEMU [10] e LLVM [11] como ferramentas de referência.

IV. ARQUITETURA PROPOSTA

A. Escopo do processador

O núcleo modelado é RV64 com as extensões I, M e Zicsr, um subconjunto da RVV 1.0 (configuração de vetor, cargas e armazenamentos unitários e indexados, FP32 com `vfmul` e `vfredusum`) e Zvtakum. Não há modo privilegiado, MMU, instruções comprimidas nem cache. Os contadores medem o que o programa executa, não o tempo.

B. Codificação

As instruções Zvtakum usam o espaço custom-0 (`opcode = 0x0B`), reservado pela ISA para extensões não padronizadas. O campo `funct6` seleciona a operação, `vm` (bit 25) segue a convenção da RVV e `funct3` vale zero (Tabela I).

A versão anterior do projeto usava o espaço OP-V com `funct6 = 0b111001`, sob a premissa de que seria o único valor livre. O inventário do riscv-opcodes [12] mostra que esse valor está livre, mas há entre 11 e 33 outros valores livres por classe de operando. A premissa não se sustenta, e a codificação foi migrada.

C. Semântica

Todas as instruções respeitam `vl`, `vstart`, a máscara `v0` e as políticas `vta` e `vma`. Com `vl = 0`, não há elementos ativos, e os elementos de cauda seguem `vta`. Elementos agnósticos recebem 1s. SEW vem de `vtype` (8, 16 ou 32), e SEW $= 64$ é ilegal nas instruções Zvtakum. O acumulador de `vtdot.vv` tem EEW $= 2$·SEW e ocupa um único registrador; só o elemento 0 é escrito, e `vstart` diferente de zero é ilegal. A origem de `vncvt.t.t` é um grupo de EMUL $= 2$·LMUL alinhado. Não há modo de arredondamento nem flags acumuladas: as instruções Takum não leem `frm` nem gravam `fflags`.

D. Arredondamento

O arredondamento é feito no código, e não no valor. Para um valor exato não nulo, o simulador monta a cadeia infinita do código, toma o prefixo de $n + 1$ bits e o bit *sticky*, que indica bits não nulos depois do prefixo. Se o último bit do prefixo é 1 e o sticky está ativo, ou se o código de n bits é ímpar, o resultado é arredondado para cima. O resultado é limitado a $[1, \mathrm{maxpos}]$, e os negativos recebem o complemento de dois. Essa regra coincide com o modelo Python para T8 completo e para amostras de T16, T32 e T64. Ela pode diferir do vizinho mais próximo no valor, como registrado em `docs/devlog.md`.

E. Unidade Takum

A decodificação lê a janela de 12 bits, obtém S, D, R e C, e extrai a fração por um deslocamento de largura variável. O valor é representado por expoente, significando inteiro normalizado e bit sticky. A soma alinha o menor operando com sticky, soma ou subtrai e normaliza. O produto multiplica os significandos em até 28 bits por 28 bits (T32) ou 60 bits por 60 bits (T64), com aritmética de 128 bits. O `vtdot` arredonda o produto em $2n$ e depois a soma em $2n$. O estreitamento decodifica exatamente e codifica no formato menor. O uso de `double` foi evitado de propósito: as 128 divergências registradas no modelo Python contra a libtakum vêm de arredondamento duplo nesse intermediário.

V. IMPLEMENTAÇÃO E VERIFICAÇÃO

A. Simulador

O simulador `zt-sim` tem três partes: a aritmética Takum (`takum.cpp`), o núcleo com escalar, CSRs, RVV e Zvtakum (`machine.cpp`) e a linha de comando, que carrega ELF64 RISC-V e atende `write` e `exit` como o QEMU. Os contadores registram instruções por classe, escritas de CSR (incluindo `vl` e `vtype` escritos por `vsetvl`), elementos ativos, bytes lidos e escritos e bytes de índice.

B. Níveis de verificação

1) **Núcleo Takum.** Tabelas T8 completas de soma, subtração e produto; narrow completo de T16 para T8; operações amostradas de T16, T32 e T64; `vtdot` amostrado; ida e volta do codificador. São 21 verificações, todas aprovadas, contra o modelo Python.
2) **Escalar e RVV padrão.** Dois programas: operações escalares, com 28 856 bytes de saída, e RVV padrão, com 342 instruções por execução. O RVV inclui uma redução com o elemento 0 mascarado, políticas de cauda e máscara `tu/mu` e `ta/ma`, `vl = 0` e `vstart` escrito pelo programa. Os dois são executados em VLEN de 128, 256 e 512 bits, e a saída e o código de saída são comparados com o QEMU 10.2.2 em modo usuário, com `vext_spec=v1.0`. O QEMU é executado com `rvv_ta_all_1s` e `rvv_ma_all_1s`, para que os

elementos agnósticos recebam 1s, como no simulador. A RVV 1.0 permite as duas escolhas; sem essa opção, as diferenças aparecem apenas nos elementos agnósticos. As seis execuções são iguais.

3) **Zvtakum.** Um programa com 101 instruções, 10 delas Zvtakum: as cinco operações, com máscara `v0`, cauda, `vl` parcial, `vl = 0`, grupos LMUL$=2$ e a sequência `vtdot`/`vncvt`. A saída esperada vem do modelo Python. As três execuções (uma por VLEN) são iguais.
4) **Kernels SpMV.** Cada um dos 96 kernels (oito matrizes, quatro formatos, três VLENs) é comparado com `scalar_spmv`, que arredonda cada produto e cada soma no formato de acumulação, na mesma ordem. As 96 execuções são bit a bit iguais.
5) **Modelo de contagem.** Os parâmetros do modelo da Seção VII-D são medidos com 12 programas sintéticos e conferidos contra as 96 execuções reais. Todas as 12 verificações sintéticas passam, e o resíduo máximo nas execuções reais é 0.

VI. Metodologia experimental

As matrizes são oito da coleção Harwell–Boeing [13], disponíveis em `python/data/harwell_boeing`: `bcsstk01`, `bcsstk02`, `jpwh_991`, `pores_1`, `sherman1`, `well1033`, `west0067` e `west0132`. Têm entre 30 e 1033 linhas e até 6027 não nulos. O vetor x é pseudoaleatório (semente 2026), com elementos $k/2^{52}$ e $|k| \le 2^{52}$, para forçar arredondamento em FP32 e em takum32. Uma versão anterior, com elementos de 20 bits, dava erro zero em FP32 e em takum32 para `jpwh_991`: os valores da matriz são inteiros pequenos, e para essas magnitudes qualquer múltiplo de 2^{-20} cabe exatamente em FP32. Esse caso foi descartado como teste fraco.

Os kernels seguem o mesmo laço escalar sobre as linhas do CSR, com faixas (*strip-mining*) de VLMAX elementos. O kernel FP32 usa `vfmul.vv` e `vfredusum.vs`, com acumulador de 32 bits. O kernel takum usa `vtdot.vv` com acumulador de $2n$ bits e `vncvt.t.t` no fim de cada linha. Os dois usam o gather `vluxei32` para x, com índices de 32 bits. Os deslocamentos de x, pré-multiplicados pelo tamanho do elemento, são preparados no host e são iguais para formatos do mesmo tamanho. Todo o restante executa no simulador, inclusive os `vsetvli` e a zeragem do acumulador.

O erro é o erro relativo L2 em relação à referência binary128 (113 bits de significando), que arredonda cada produto e cada soma. Essa referência é um modelo de software validado contra o `__float128` do GCC, com 6 014 de 6 017 resultados iguais em bits; as três diferenças são de sinal de zero, previstas no contrato do modelo. Os controles de acumulação repetem a mesma matriz e o mesmo x, com FP32 e takum32 acumulando na largura do formato e em largura ampliada. E4M3, E5M2 e FP16 existem apenas como modelo numérico, sem contagem de instruções.

As comparações entre formatos são pareadas por matriz: contamos em quantas matrizes cada formato tem erro menor. Medianas servem apenas para resumir uma coluna, nunca para comparar conjuntos de matrizes diferentes. Com um único vetor x e oito matrizes, os resultados devem ser lidos como ordens de grandeza, sem intervalos de confiança.

VII. Resultados e discussão

A. Corretude

Todas as verificações da seção anterior passaram. As 96 execuções dos kernels produziram saída bit a bit igual ao modelo de referência.

B. Precisão em 32 bits

A Tabela II mostra o erro por matriz. A mediana do erro do kernel FP32 é $5{,}91 \times 10^{-8}$, e a do kernel Takum32 é $1{,}15 \times 10^{-8}$. Pareado por matriz, o Takum32 tem erro menor em 7 de 8 matrizes; a exceção é `pores_1`.

A diferença entre os kernels mistura dois efeitos: o FP32 acumula em 32 bits, e o Takum32 acumula em 64 bits. A razão entre as medianas, de 5,15, se decompõe em 2,01, o efeito do formato com o mesmo acumulador de 32 bits nos dois casos, e 2,56, o efeito do acumulador de 64 bits do `vtdot`. Com acumulador de 32 bits, o Takum32 tem erro menor em 7 de 8 matrizes. Com acumulador de 64 bits nos dois casos, o FP32 tem mediana de $3{,}61 \times 10^{-8}$, e o Takum32 é 3,15 vezes melhor. O efeito de formato é, portanto, real, mas menor que a diferença entre os kernels.

C. Precisão em 8 e 16 bits

A Tabela III mostra os formatos de 8 e 16 bits. Takum8 e Takum16 não produzem resultados não finitos em nenhuma das oito matrizes. Em contraste, E4M3 tem overflow em 4 matrizes, E5M2 em 3 e FP16 em 3. Esses resultados não finitos não são números e não entram em nenhuma comparação de erro.

Nas matrizes em que ambos os formatos são finitos, o resultado depende do par comparado:

- Takum16 tem erro menor que FP16 (ambos com acumulador de 32 bits) em 3 de 5 matrizes, e FP16 é melhor nas 2 restantes;
- Takum8 tem erro menor que E5M2 (ambos com acumulador de 16 bits) em 3 de 5 matrizes, e E5M2 é melhor nas 2 restantes;
- E4M3 tem erro menor que Takum8 em 4 de 4 matrizes.

Em 8 bits, portanto, o Takum troca precisão por ausência de overflow: nunca estoura, mas perde para E4M3 nas matrizes em que E4M3 não estoura.

D. Modelo de contagem de instruções

O número de instruções de cada kernel segue

$$N_f = r_f\, R + c_f\, S + K, \tag{1}$$

em que R é o número de linhas da matriz, S é o número de faixas somado sobre as linhas não vazias e K é a preparação e o encerramento comuns. O custo r_f é por linha e c_f é por faixa. A Tabela IV mostra os parâmetros medidos. Eles reproduzem exatamente as 96 execuções reais e as 12 verificações sintéticas.

Tabela II
ERRO RELATIVO L2 DO SPMV EM 32 BITS (VLEN = 256), POR MATRIZ. A COLUNA FP32 É O KERNEL VETORIAL (ACUMULADOR DE 32 BITS); AS COLUNAS DE CONTROLE USAM O MODELO NUMÉRICO COM O ACUMULADOR INDICADO.

Matriz	nnz	FP32	Takum32	FP32 (acum. 64)	Takum32 (acum. 32)
bcsstk01	400	$7{,}20 \times 10^{-8}$	$5{,}06 \times 10^{-8}$	$3{,}57 \times 10^{-8}$	$6{,}22 \times 10^{-8}$
bcsstk02	4356	$1{,}81 \times 10^{-7}$	$1{,}88 \times 10^{-8}$	$4{,}63 \times 10^{-8}$	$8{,}32 \times 10^{-8}$
jpwh_991	6027	$6{,}32 \times 10^{-8}$	$4{,}19 \times 10^{-9}$	$3{,}47 \times 10^{-8}$	$9{,}40 \times 10^{-9}$
pores_1	180	$3{,}54 \times 10^{-8}$	$7{,}87 \times 10^{-8}$	$3{,}26 \times 10^{-8}$	$5{,}44 \times 10^{-8}$
sherman1	3750	$5{,}82 \times 10^{-8}$	$4{,}02 \times 10^{-9}$	$3{,}65 \times 10^{-8}$	$4{,}93 \times 10^{-9}$
well1033	4732	$5{,}99 \times 10^{-8}$	$4{,}07 \times 10^{-9}$	$3{,}72 \times 10^{-8}$	$6{,}45 \times 10^{-9}$
west0067	294	$4{,}82 \times 10^{-8}$	$3{,}59 \times 10^{-9}$	$3{,}27 \times 10^{-8}$	$4{,}84 \times 10^{-9}$
west0132	413	$5{,}01 \times 10^{-8}$	$3{,}21 \times 10^{-8}$	$4{,}00 \times 10^{-8}$	$4{,}94 \times 10^{-8}$
Mediana		$5{,}91 \times 10^{-8}$	$1{,}15 \times 10^{-8}$	$3{,}61 \times 10^{-8}$	$2{,}94 \times 10^{-8}$

Tabela III
ERRO RELATIVO L2 EM 8 E 16 BITS, POR MATRIZ (VLEN = 256). NF(k) INDICA k LINHAS NÃO FINITAS (OVERFLOW). ACUMULADORES: E4M3, E5M2 E TAKUM8 EM 16 BITS; FP16 E TAKUM16 EM 32 BITS.

Matriz	E4M3	E5M2	FP16	Takum16	Takum8
bcsstk01	NF(48)	NF(48)	NF(48)	$3{,}24 \times 10^{-3}$	$6{,}44 \times 10^{-1}$
bcsstk02	NF(66)	$7{,}31 \times 10^{-2}$	$3{,}76 \times 10^{-4}$	$1{,}20 \times 10^{-3}$	$2{,}27 \times 10^{-1}$
jpwh_991	$3{,}45 \times 10^{-2}$	$7{,}58 \times 10^{-2}$	$2{,}77 \times 10^{-4}$	$3{,}04 \times 10^{-4}$	$7{,}77 \times 10^{-2}$
pores_1	NF(30)	NF(15)	NF(15)	$1{,}18 \times 10^{-3}$	$8{,}58 \times 10^{-1}$
sherman1	$3{,}95 \times 10^{-2}$	$1{,}00 \times 10^{-1}$	$3{,}06 \times 10^{-4}$	$2{,}28 \times 10^{-4}$	$6{,}40 \times 10^{-2}$
well1033	$4{,}24 \times 10^{-2}$	$8{,}75 \times 10^{-2}$	$3{,}54 \times 10^{-4}$	$2{,}62 \times 10^{-4}$	$6{,}47 \times 10^{-2}$
west0067	$3{,}43 \times 10^{-2}$	$8{,}74 \times 10^{-2}$	$3{,}36 \times 10^{-4}$	$1{,}76 \times 10^{-4}$	$5{,}18 \times 10^{-2}$
west0132	NF(17)	NF(3)	NF(3)	$1{,}40 \times 10^{-3}$	$1{,}21 \times 10^{0}$

Tabela IV
PARÂMETROS DO MODELO DE CONTAGEM $N = rR + cS + K$, EM QUE R É O NÚMERO DE LINHAS E S O NÚMERO DE FAIXAS DAS LINHAS NÃO VAZIAS. A ÚLTIMA COLUNA COMPARA O MODELO COM AS EXECUÇÕES REAIS.

Formato	r	c	K	Sintéticos	Reais exatas
FP32	13	12	21	3/3	24/24
Takum8	14	12	21	3/3	24/24
Takum16	14	13	21	3/3	24/24
Takum32	14	11	21	3/3	24/24

A diferença do Takum32 para o FP32 sai diretamente dos parâmetros. Cada linha do Takum32 custa uma instrução a mais, o `vncvt.t.t` de fechamento, e cada faixa custa uma a menos, pois o `vtdot` substitui `vfmul` e `vfredusum`. Logo

$$N_{\text{Takum32}} - N_{\text{FP32}} = R - S. \tag{2}$$

A identidade vale em todos os 24 pares matriz×VLEN, e os empates ocorrem exatamente quando $S = R$. Assim, o Takum32 executa menos instruções que o FP32 em 13 pares, é igual em 11, e em nenhum par executa mais. Um caso com mais instruções é possível: basta haver linhas vazias e nenhuma linha com mais de VLMAX elementos, porque então $R > S$.

Os kernels de 8 e 16 bits diferem do FP32 no custo por faixa. O kernel de 8 bits tem $c = 12$, igual ao FP32: o `vtdot` economiza uma instrução por faixa, e a configuração extra do índice de 32 bits gasta outra. O de 16 bits tem $c = 13$, porque também calcula um deslocamento a mais para os valores de 2 bytes. Com S_b o número de faixas para elementos de b bits, a diferença em relação ao FP32 é $R + 12\,(S_8 - S_{32})$ para Takum8 e $R + 13\,S_{16} - 12\,S_{32}$ para Takum16. Como $S_8 \le S_{32}$ e $S_{16} \le S_{32}$, a redução de faixas compensa o custo de uma instrução por linha somente enquanto as linhas precisam de várias faixas. Em VLEN grande, quase todas as linhas cabem em uma faixa, a redução desaparece e resta o custo por linha. Isso explica por que o Takum8 passa de uma economia para um custo quando o VLEN cresce.

E. Custo de instruções e escritas de CSR

A Tabela V mostra instruções, instruções Zvtakum, escritas de CSR e elementos ativos por elemento não nulo. Em VLEN de 256 bits, o kernel FP32 executa 4,59 instruções por não nulo, e o kernel Takum32 executa 4,55, uma diferença de $-0{,}7\%$. A diferença é de $-2{,}7\%$ em VLEN de 128 bits e de $-0{,}3\%$ em VLEN de 512 bits. As escritas de CSR são as mesmas: 1,07 por não nulo para FP32 e 1,07 para Takum32, em VLEN de 256 bits. Os dois kernels usam um `vsetvli` por faixa e dois por linha.

Os kernels de 8 e 16 bits executam duas configurações vetoriais por faixa, e isso aumenta as escritas de CSR por não nulo: +27,4% para Takum8 e +29,9% para Takum16, em VLEN de 256 bits. Em instruções, a diferença vai de $-26{,}6\%$ (Takum8, VLEN de 128 bits) a +1,1% (VLEN de 512 bits), e para Takum16 de $-19{,}3\%$ a +6,0%. O kernel Takum32 processa menos elementos por não nulo, pois o `vtdot` conta um elemento ativo por produto, enquanto `vfmul` e `vfredusum` contam dois. Os valores são 4,50 para Takum32 e 5,33 para FP32.

Tabela V
INSTRUÇÕES, INSTRUÇÕES ZVTAKUM, ESCRITAS DE CSR E ELEMENTOS ATIVOS POR ELEMENTO NÃO NULO, SOMADOS SOBRE AS OITO MATRIZES. ELEMENTOS ATIVOS CONTAM CADA ELEMENTO PROCESSADO POR INSTRUÇÕES VETORIAIS DE DADOS (CARGA, GATHER, PRODUTO, REDUÇÃO E ARMAZENAMENTO).

Formato	VLEN	Instr./nnz	Zvtakum/nnz	Escritas CSR/nnz	Elem. ativos/nnz
FP32	128	6,140	0,000	1,328	5,334
FP32	256	4,587	0,000	1,069	5,334
FP32	512	4,343	0,000	1,029	5,334
Takum32	128	5,977	0,497	1,328	4,501
Takum32	256	4,553	0,368	1,069	4,501
Takum32	512	4,329	0,347	1,029	4,501
Takum16	128	4,954	0,368	1,470	4,501
Takum16	256	4,690	0,347	1,389	4,501
Takum16	512	4,605	0,341	1,363	4,501
Takum8	128	4,510	0,347	1,389	4,501
Takum8	256	4,431	0,341	1,363	4,501
Takum8	512	4,392	0,337	1,350	4,501

F. Discussão

Três resultados sustentam as conclusões. Primeiro, com acumuladores de mesma largura, o Takum32 tem erro cerca de 2,01 vezes menor que o FP32 na mediana, e vence em 7 de 8 matrizes. Segundo, o Takum32 não custa mais instruções que o FP32 em nenhum dos pares testados, e custa as mesmas escritas de CSR; a economia é pequena, de até 2,7% em VLEN de 128 bits, e vem do `vtdot`, que une o produto e a redução, descontado o estreitamento de cada linha. Terceiro, em 8 e 16 bits, o Takum evita overflow, mas não é consistentemente mais preciso que os formatos IEEE-like, e o kernel de 8 e 16 bits paga uma configuração vetorial a mais por faixa.

A conclusão defensável é que o Takum32 é uma alternativa de baixo custo ao FP32 para SpMV neste subconjunto de instruções: tem erro menor em 7 de 8 matrizes com acumulador de mesma largura, e custo de instruções igual ou menor. Os ganhos de 8 e 16 bits dependem da matriz e do VLEN.

VIII. LIMITAÇÕES

O trabalho tem limitações que afetam a interpretação dos resultados. As oito matrizes são pequenas e não representam a coleção SuiteSparse, e há um único vetor x; os erros devem ser lidos como ordens de grandeza. O estudo de 1401 matrizes com MuFoLAB não foi executado, porque exige Julia e uma rodada longa, e os resultados de representação da versão anterior, com 100 matrizes e referência `long double`, não foram substituídos. A variante logarítmica não existe nesta versão, e o custo da soma logarítmica ($\Delta\pm$) não foi medido. A FP16 vetorial (Zvfh) é reconhecida pelo montador, mas não é executada pelo simulador. FP8 não existe na RVV, e seus números são apenas numéricos. Os contadores medem instruções e eventos, não ciclos, e não há estimativa de área ou energia. A conformidade é verificada por amostra e por casos de borda, não por prova. A comparação com o QEMU depende de preencher os elementos agnósticos com 1s, escolha que a especificação permite e que o simulador adota. Por fim, o kernel usa um laço simples; uma versão que evite a configuração extra do índice de 32 bits mudaria os números de 8 e 16 bits.

Trabalhos futuros incluem executar o estudo de 1401 matrizes, implementar Zvfh e o kernel FP16 com acumulação ampliada, decidir a variante logarítmica e medir o custo de $\Delta\pm$, reduzir as trocas de configuração do kernel de 8 e 16 bits e comparar com uma implementação em hardware.

IX. CONCLUSÃO

Este trabalho propôs a extensão Zvtakum sobre a RVV 1.0 para T8, T16 e T32, com codificação em custom-0, semântica especificada e uma aritmética Takum em inteiros. O simulador funcional executa RV64IM, Zicsr, RVV e Zvtakum e foi verificado contra o QEMU e contra um modelo de racionais exatos. Nos kernels SpMV, o Takum32 tem erro menor que o FP32 em 7 de 8 matrizes com acumulador de mesma largura, e nunca executa mais instruções. O custo de instruções segue um modelo analítico, $N = rR + cS + K$, que reproduz todas as execuções. Em 8 e 16 bits, o Takum não estoura, mas não é consistentemente mais preciso que os formatos IEEE-like, e o custo depende do VLEN.

REFERÊNCIAS

[1] RISC-V International, "The RISC-V vector extension, version 1.0," https://github.com/riscv/riscv-v-spec, 2021.

[2] P. Micikevicius, S. Oberman, P. Dubey *et al.*, "OCP 8-bit floating point specification (OFP8)," Open Compute Project, 2023.

[3] S. Wang and P. Kanwar, "BFloat16: The secret to high performance on cloud TPUs," 2019, google Cloud Blog.

[4] L. Hunhold, "Beating posits at their own game: Takum arithmetic," in *Next Generation Arithmetic (CoNGA 2024), LNCS vol. 14666*. Springer Nature Switzerland, 2024, arXiv:2404.18603.

[5] ——, "Streamlining SIMD ISA extensions with takum arithmetic: A case study on intel AVX10.2," in *2025 14th International Conference on Modern Circuits and Systems Technologies (MOCAST)*, 2025, pp. 1–6, arXiv:2503.14067.

[6] ——, "Design and implementation of a takum arithmetic hardware codec in VHDL," 2024, arXiv:2408.10594.

[7] J. L. Gustafson and I. Yonemoto, "Beating floating point at its own game: Posit arithmetic," *Supercomputing Frontiers and Innovations*, vol. 4, no. 2, pp. 71–86, 2017.

[8] L. Hunhold, "libtakum: Takum arithmetic C99 reference implementation, v2.0.0," https://github.com/takum-arithmetic/libtakum, 2025.

[9] L. Hunhold and J. Quinlan, “MuFoLAB – multi-format linear algebra benchmarks,” 2025.
[10] “QEMU: emulador de processador e máquina, modo usuário,” https://www.qemu.org, versão 10.2.2, CPU rv64 com extensão vetorial RVV 1.0.
[11] “LLVM mc: montador e desmontador para risc-v,” https://llvm.org, versão 23.1.3.
[12] “riscv-opcodes: inventário de codificações risc-v,” https://github.com/riscv/riscv-opcodes.
[13] I. S. Duff, R. G. Grimes, and J. G. Lewis, “Sparse matrix test problems,” *ACM Transactions on Mathematical Software*, vol. 15, no. 1, pp. 1–14, 1989.